

INFORME FINAL

Mato Seihei no Slave — Battle Arena OO

Trabajo práctico de Diseño Orientado a Objetos / ADOO-PDS

Versión final documentada

Java 17 · Swing · Web HTML/CSS/JavaScript · Persistencia · Patrones · GRASP · SOLID
· MVC · Observer

Este informe detalla la arquitectura, modelo, funcionalidades, patrones, contenidos académicos, interfaces, persistencia, multimedia y verificaciones de la versión final del proyecto.

Fecha de emisión: 4 de octubre de 2026

Índice

Índice	2
1. Resumen ejecutivo	4
2. Objetivo y alcance	4
2.1 Objetivo general	4
2.2 Alcance funcional	4
3. Arquitectura general	4
3.1 Organización por paquetes	4
3.2 Flujo de alto nivel	5
3.3 Diagrama de clases	5
4. Modelo de dominio	5
4.1 Entidad Waifu	5
4.2 Equipos y Composite	5
4.3 Estados	5
5. Roster final y estadísticas	6
6. Habilidades especiales	6
7. Sistema de combate	6
7.1 Turno del jugador	6
7.2 Turno del oponente	7
7.3 Curación y finalización	7
8. Casos de uso	7
9. Patrones de diseño aplicados	7
10. GRASP y SOLID	8
10.1 GRASP	8
10.2 SOLID	8
11. MVC y Observer	8
11.1 MVC	8

11.2 Observer	8
12. Persistencia y guardado	9
13. Multimedia, imágenes e interfaces	9
13.1 Multimedia	9
13.2 Imágenes	9
13.3 Interfaces	9
14. Documentación académica incluida	9
15. IA aplicada al diseño OO	10
16. Verificación técnica	10
16.1 Java	10
16.2 JavaScript	10
16.3 Recursos gráficos	10
17. Entrega portable	10
18. Code smells y refactorizaciones	11
19. Matriz de trazabilidad	11
20. Conclusión	12
Anexo A. Estructura resumida	12
Anexo B. Ficha técnica final	12

1. Resumen ejecutivo

La versión final presenta un juego de combate orientado a objetos basado en el universo de Mato Seihei no Slave. Integra personajes, equipos, estados, estrategias, habilidades especiales, observers, persistencia, multimedia y dos interfaces de usuario.

El elenco final contiene ocho capitanas: Konomi Tatara, Bell Tsukiyono, Yakumo Ezo, Tenka Izumo, Kyouka Uzen, Varvara Pilipenko, Fubuki Azuma y Ren Yamashiro. Mira Kamiunten fue eliminada del código, datos, interfaz web y recursos gráficos; Riu Myouga no forma parte del elenco.

Desde el punto de vista académico, el proyecto reúne modelado OO, casos de uso, secuencias, GRASP, SOLID, code smells/refactorización, MVC, Strategy, Adapter, Composite, Decorator, Facade, State, Observer, Singleton, Factory, DAO, Repository, Table Data Gateway e IA aplicada al diseño OO.

Resultado: una solución modular donde las variaciones de comportamiento están encapsuladas mediante interfaces, composición y polimorfismo.

2. Objetivo y alcance

2.1 Objetivo general

Construir una aplicación de combate que permita demostrar, dentro de un sistema funcional, conceptos de diseño orientado a objetos y patrones de diseño estudiados en la materia.

2.2 Alcance funcional

- Selección de 1 a 3 personajes por equipo y validación de que no estén en ambos equipos.
- Selección explícita de atacante, objetivo y estrategia por parte del jugador.
- Tres estrategias: agresiva, equilibrada y defensiva.
- Una habilidad especial diferente por capitana y de uso único por combate.
- Curación de hasta 35 HP.
- Turno automático del oponente: mayor ATK como atacante y menor HP como objetivo.
- Estados saludable, herida y KO.
- Registro de eventos mediante Observer y logro al tercer evento.
- Guardado/continuación del combate.
- Opening y ending automáticos, con opción de saltar.
- Aplicación Java Swing y versión web portable.

3. Arquitectura general

3.1 Organización por paquetes

Paquete	Responsabilidad
model	Entidades, estados, composición y decoradores del dominio.
strategy	Algoritmos intercambiables de cálculo de daño.
ability	Habilidades especiales y Factory.
factory	Creación de personajes.
facade	Punto de entrada simplificado al dominio.

3.2 Flujo de alto nivel

AnimeGameFacade evita que la vista deba conocer simultáneamente Factory, Adapter, Repository, estrategias y composición de equipos.

3.3 Diagrama de clases



4. Modelo de dominio

4.1 Entidad Waifu

Sus operaciones principales incluyen `receiveDamage()`, `heal()`, `useAbility()`, `resetBattleState()` y `restoreBattleState()`. Los cambios relevantes notifican a los observers.

4.2 Equipos y Composite

4.3 Estados

Página 5

Estado	Comportamiento
KnockedOutState	No puede atacar y queda KO.

El estado se determina a partir del HP: 0 produce KO; hasta la mitad del máximo produce Herida; por encima de ese umbral, Saludable.

5. Roster final y estadísticas

El elenco activo contiene exactamente ocho capitanas. Las imágenes y datos se encuentran alineados con este roster.

#	Capitana	Elemento	ATK	DEF	HP	Habilidad
1	Konomi Tatara	WIND	82	45	185	BEAST POWER
2	Bell Tsukiyono	LIGHT	78	48	190	CANOPUS
3	Yakumo Ezo	WIND	88	52	195	NIGHT STORM
4	Tenka Izumo	LIGHT	84	58	205	AME-NO-MITORI
5	Kyouka Uzen	SHADOW	86	55	210	SLAVE
6	Varvara Pilipenko	SHADOW	80	70	220	PANTHEON
7	Fubuki Azuma	WATER	79	64	215	SUNSET
8	Ren Yamashiro	FIRE	94	68	225	ALL-ENCOMPASSING LORD OF THE COSMOS

ImageRepository convierte los nombres a slugs como konomi-tatara.jpg, permitiendo resolver correctamente los recursos de Swing desde el classpath.

6. Habilidades especiales

Cada capitana posee una implementación concreta de SpecialAbility y SpecialAbilityFactory asocia el nombre con su habilidad. Waifu controla que una habilidad solo pueda usarse una vez por combate.

Capitana	Habilidad	Efecto
Konomi Tatara	BEAST POWER	+35 ATK
Bell Tsukiyono	CANOPUS	+25 ATK y +20 DEF
Yakumo Ezo	NIGHT STORM	+40 ATK
Tenka Izumo	AME-NO-MITORI	+30 ATK y +15 DEF
Kyouka Uzen	SLAVE	+45 ATK
Varvara Pilipenko	PANTHEON	+40 DEF
Fubuki Azuma	SUNSET	Recupera hasta 55 HP
Ren Yamashiro	ALL-ENCOMPASSING LORD OF THE COSMOS	+30 ATK y +25 DEF

La versión web reproduce estas ocho habilidades mediante activateAbility(), manteniendo la correspondencia funcional con Java.

7. Sistema de combate

7.1 Turno del jugador

El jugador selecciona atacante, objetivo y estrategia. CombatStrategy encapsula el cálculo de daño.

Estrategia	Regla
AGRESIVA	$ATK + 20 - DEF \text{ objetivo} / 3$
EQUILIBRADA	$ATK + DEF \text{ atacante} / 5 - DEF \text{ objetivo} / 2$
DEFENSIVA	$ATK - DEF \text{ objetivo} / 2$

El daño se limita a un mínimo de 1.

7.2 Turno del oponente

El sistema selecciona la enemiga viva con mayor ATK y la aliada viva con menor HP. Si la atacante aún tiene habilidad disponible, la usa una vez. Luego elige una estrategia en función de sus estadísticas y ejecuta el ataque.

7.3 Curación y finalización

Curar recupera hasta 35 HP sin superar el máximo. El combate finaliza cuando un equipo queda sin personajes vivos. La web calcula además un puntaje final basado en HP restante y daño producido.

8. Casos de uso

ID	Caso de uso	Resultado
CU-01	Iniciar combate	Crea dos equipos válidos, reinicia el estado y comienza la batalla.
CU-02	Atacar con una waifu	Calcula daño, modifica HP/estado y notifica el evento.
CU-03	Utilizar habilidad especial	Aplica la habilidad única y consume su uso.
CU-04	Turno del oponente	Selecciona atacante/objetivo automáticamente y ejecuta el ataque.
CU-05	Curar personaje	Recupera HP hasta el máximo.
CU-06	Guardar / continuar combate	Persiste equipos, turno, selección, estrategia, HP, habilidades, log y daño.
CU-07	Finalizar combate	Determina victoria/derrota y reproduce el ending.

9. Patrones de diseño aplicados

La versión final incorpora doce patrones explícitamente documentados e implementados:

Patrón	Implementación	Aporte
Strategy	CombatStrategy + 3 estrategias	Algoritmos alternativos de daño.
Adapter	AnimeApiAdapter	Convierte datos externos a Waifu.
Composite	Squad / SquadMember	Uniformidad entre equipo y miembro.
Decorator	WaifuDecorator / PowerUpDecorator / ArmorDecorator	Agrega modificadores sin alterar Waifu.
Facade	AnimeGameFacade	Simplifica la coordinación de la aplicación.
State	WaifuState + 3 estados	Comportamiento dependiente del estado.
Observer	Subject/Observer + observers	Difunde eventos a UI, consola y logros.
Singleton	DatabaseConnection	Instancia compartida de conexión simulada.
Factory	WaifuFactory + SpecialAbilityFactory	Centraliza creación.
DAO	WaifuDAO + InMemoryWaifuDAO	Abstrae acceso a datos.

Patrón	Implementación	Aporte
Repository	WaifuRepository + Impl	Abstrae persistencia para la aplicación.
Table Data Gateway	WaifuTableDataGateway	Representa la tabla en memoria.

10. GRASP y SOLID

10.1 GRASP

Principio	Evidencia
Controller	WaifuController y WaifuGameFrame reciben eventos y delegan.
Creator	Factories conocen las reglas de creación.
Information Expert	Waifu administra HP, ATK, DEF, estado y habilidad.
Low Coupling	Facade trabaja con abstracciones.
High Cohesion	Paquetes separados por responsabilidad.
Polymorphism	Strategy, State, Observer, SpecialAbility, Decorator y Composite.
Pure Fabrication	Repository, DAO, Gateway, MediaLauncher y factories.
Indirection	Facade, Adapter, Repository y DAO.
Protected Variations	Interfaces protegen variaciones de estrategias, estados, habilidades y fuentes externas.

10.2 SOLID

Principio	Aplicación
SRP	Separación de dominio, factories, multimedia, persistencia, adapter y estrategias.
OCP	Nuevas estrategias, estados, observers y habilidades por extensión.
LSP	Implementaciones sustituibles por sus contratos.
ISP	Interfaces pequeñas y enfocadas.
DIP	Dependencia de abstracciones como WaifuRepository y CombatStrategy.

11. MVC y Observer

11.1 MVC

Model: Waifu, Squad, estados, habilidades, estrategias, decoradores y persistencia.

View: ConsoleView, WaifuGameFrame e interfaz web.

Controller: WaifuController y eventos de las interfaces; AnimeGameFacade actúa como límite simplificado de aplicación.

11.2 Observer

- ConsoleNotificationObserver: eventos en consola.
- UiNotificationObserver: eventos en el registro visual.
- AchievementObserver: desbloquea un logro al tercer evento.

En web se reproduce el concepto con BattleSubject, UiObserver y AchievementObserver. Waifu no necesita conocer directamente la UI ni el sistema de logros.

12. Persistencia y guardado

Mecanismo	Implementación	Finalidad
Persistencia de waifus	Repository → DAO → Table Data Gateway	Guardar/recuperar entidades en memoria.
Conexión	DatabaseConnection (Singleton)	Representar conexión compartida simulada.
Partida Swing	BattleSaveManager + .properties	Guardar HP, turno, equipos, selección, estrategia, log, daño y habilidades.
Partida Web	localStorage	Continuar el combate en el navegador.

No se utiliza una base de datos externa: la infraestructura de datos es deliberadamente simulada para demostrar las abstracciones de persistencia.

13. Multimedia, imágenes e interfaces

13.1 Multimedia

MediaLauncher encapsula Opening.mp4 y Ending.mp4. En Windows intenta usar Windows Media Player y ofrece un fallback mediante la aplicación predeterminada.

En web, SALTAR detiene el video, elimina el src y ejecuta load() antes de comenzar el combate, evitando que el medio continúe reproduciéndose.

13.2 Imágenes

La versión final contiene ocho JPG para las capitanas. ImageRepository centraliza carga, cache y escalado en Swing; la web utiliza las mismas denominaciones en sus recursos.

Recurso	Uso
src/main/resources/images/*.jpg	Imágenes Swing.
src/main/resources/web/images/*.jpg	Imágenes web.
portable/web/images/*.jpg	Copia portable web.
ImageRepository	Carga y escala recursos.

13.3 Interfaces

Swing incluye pantalla inicial, selección de equipos, combate, estrategias, habilidades, curación, log y resultado. La web replica el flujo mediante tarjetas, reproductor, combate, log y localStorage.

14. Documentación académica incluida

Archivo	Contenido
01_CASOS_DE_USO.md	Actores y siete casos de uso.
02_DIAGRAMAS_SECUENCIA.puml	Tres secuencias: ataque, habilidad y turno enemigo.
03_GRASP.md	Nueve principios GRASP.
04_SOLID.md	Cinco principios SOLID.

Archivo	Contenido
05_CODE_SMELLS_REFACTORIZACIONES.md	Smells detectados y refactorizaciones.
06_MVC_OBSERVER.md	MVC y demostración de Observer.
07_IA_APLICADA_AL_DISEÑO_OO.md	IA como herramienta de análisis y diseño.
08_HABILIDADES_Y_COMBATE.md	Roster final, habilidades y reglas.

15. IA aplicada al diseño OO

La IA se documenta como herramienta de apoyo al análisis y diseño, no como sustituto de la decisión del desarrollador.

- Identificación de responsabilidades mezcladas y code smells.
- Propuesta de patrones para estrategias, estados, observers, factories, decorators y facade.
- Revisión crítica de las sugerencias contra el código y el cronograma de la materia.
- Selección de patrones únicamente cuando existe una variación o responsabilidad justificable.

El criterio final permanece en el desarrollador, que debe poder explicar por qué cada patrón y refactorización es pertinente.

16. Verificación técnica

16.1 Java

Se compiló la totalidad de las 59 fuentes Java con javac usando Java 17. Resultado: javac OK. Las distribuciones compiladas contienen 60 archivos .class.

16.2 JavaScript

Se ejecutó node --check sobre portable/web/app.js y el archivo fue aceptado sin errores de sintaxis.

16.3 Recursos gráficos

Recurso	Estado
Konomi Tatara	OK
Bell Tsukiyono	OK
Yakumo Ezo	OK
Tenka Izumo	OK
Kyouka Uzen	OK
Varvara Pilipenko	OK
Fubuki Azuma	OK
Ren Yamashiro	OK
Mira Kamiunten	No presente en la versión final

La Factory, Main y la aplicación web contienen únicamente las ocho capitanas finales.

17. Entrega portable

El proyecto incluye distribución portable para Windows y una versión web. Se incluyen clases compiladas, recursos, multimedia y scripts de ejecución.

Componente	Función
portable/app/classes	Clases compiladas Java y recursos.
portable/web	Aplicación web, multimedia y estilos.
JUGAR_WAIFU_NEXUS.bat	Entrada de ejecución en Windows.
run-web.bat / run-web.sh	Lanzadores de la versión web.
PORTABLE_WINDOWS.md	Indicaciones de distribución.

18. Code smells y refactorizaciones

Las principales mejoras aplicadas durante la evolución fueron:

- Extracción de MediaLauncher para aislar multimedia.
- SpecialAbility y SpecialAbilityFactory para centralizar habilidades.
- CombatStrategy para desacoplar el algoritmo de daño.
- División de la construcción de Swing en métodos por secciones.
- ImageRepository para centralizar carga y cache de imágenes.

Como mejora futura, la selección automática del enemigo podría extraerse a una EnemyCombatPolicy y la representación textual de estrategias podría reemplazarse por un selector tipado. Son mejoras de diseño, no requisitos para el funcionamiento actual.

19. Matriz de trazabilidad

Contenido	Evidencia principal
Modelo OO	Waifu, Squad, estados, habilidades y decoradores.
Casos de uso	01_CASOS_DE_USO.md.
Secuencia	02_DIAGRAMAS_SECUENCIA.puml.
GRASP	03_GRASP.md + clases del dominio.
SOLID	04_SOLID.md + interfaces y composición.
Code smells	05_CODE_SMELLS_REFACTORIZACIONES.md.
MVC / Observer	06_MVC_OBSERVER.md + implementaciones.
Strategy	CombatStrategy + 3 estrategias.
Adapter	AnimeApiAdapter.
Composite	SquadComponent/Squad/SquadMember.
Decorator	WaifuDecorator + decoradores.
Facade	AnimeGameFacade.
State	WaifuState + 3 estados.
Singleton	DatabaseConnection.
Factory	WaifuFactory + SpecialAbilityFactory.
DAO	WaifuDAO + InMemoryWaifuDAO.

Contenido	Evidencia principal
Repository	WaifuRepository + implementación.
Table Data Gateway	WaifuTableDataGateway.
IA aplicada	07_IA_APLICADA_AL_DISEÑO_OO.md.
Habilidades / combate	08_HABILIDADES_Y_COMBATE.md + implementación.

20. Conclusión

La versión final de Mato Seihei no Slave — Battle Arena OO consolida el proyecto como una aplicación funcional y como una demostración de diseño orientado a objetos.

Los patrones participan en un flujo real: Strategy interviene en el ataque; State controla la capacidad de combate; Observer registra eventos; Factory crea personajes y habilidades; Repository/DAO/Gateway organizan persistencia; Decorator modifica builds; Composite modela equipos; Adapter integra una fuente externa; Facade simplifica la coordinación; Singleton centraliza la conexión simulada.

La entrega permite defender las decisiones funcionales y de diseño ante la materia: responsabilidades de clases, aplicación de GRASP/SOLID, elección de patrones, persistencia, MVC, Observer y uso de IA.

Estado final: proyecto Java 17 compilable, JavaScript validado, recursos gráficos verificados, documentación académica incorporada y distribución portable preparada.

Anexo A. Estructura resumida

src/main/java/com/waifu/

- model: dominio, estados, composición y decoradores.
- ability: habilidades y Factory.
- strategy: estrategias de combate.
- factory: creación de personajes.
- facade: coordinación de alto nivel.
- adapter/external: integración externa simulada.
- observer: notificaciones y logros.
- persistence: Singleton, DAO, Repository y Gateway.
- ui: Swing, multimedia, imágenes y guardado.
- controller/view: coordinación y salida.

docs/AD00/

Ocho documentos de apoyo académico.

portable/

Clases compiladas, recursos y aplicación web.

Anexo B. Ficha técnica final

Dato	Valor
Lenguaje principal	Java 17
Desktop	Java Swing

Dato	Valor
Web	HTML + CSS + JavaScript
Fuentes Java	59 .java
Clases compiladas	60 .class por distribución compilada
Capitanas activas	8
Imágenes JPG	8
Casos de uso	7
Patrones explícitos	12
Principios GRASP	9
Principios SOLID	5
Documentos ADOO/PDS	8
Validación Java	javac: OK
Validación JavaScript	node --check: OK